

Quantum Error Mitigation

Khushi Chaturvedi, Kelly Mae Allen, Christopher Mori, Kausthubh Prabha Chandramouli, Dr. Dror Baron
Department of Electrical and Computer Engineering, North Carolina State University

INTRODUCTION

What is the issue? Modern small scale quantum computers (known as NISQ devices) can solve some hard problems, but they also make a lot of mistakes (errors). These errors create a noisy output that we cannot trust.

Why does that matter? If we can reduce this errors after we run our quantum circuit (post-processing), we can get clearer, more accurate results without having to wait for perfect hardware.

Key insight: Wrong outputs are clustered in Hamming distance (a small number of bit flips) from the correct answer on average, rather than occurred randomly.



DISCUSSION

Comparison: HAMMER is straight-forward and effective for small errors; Q-BEEP is effective for complex error patterns.

Trade-offs: HAMMER requires fewer runs; Q-BEEP requires more data and computation.

Limitations: Both HAMMER and Q-BEEP assume constant error rates, meaning dynamic drift of the device is not yet considered.

Method

Simulation Setup:

Tools & Platform

- Qiskit (Python) to create and transpile circuits.
 - Hazel HPC cluster to run for batch submission (use multi-core compute nodes for larger batch runs).
 - Quiet Simulations: Run the quantum circuit in Qiskit's Aer simulator (noise free) to obtain the true output distribution $P_{\text{true}}(x)$.
 - Noisy Simulation: Using Qiskit's noise models (IBM calibration data) run on the compute nodes of Hazel to create $P_{\text{noisy}}(x)$.
 - Data Capture: Automate and submit jobs to Hazel: batch runs of approx 10,000 shots per circuit, with counts collected from both modes of simulation in CSV files for easy processing in the next stage.
- Quiet Simulation: Run the quantum circuit in an ideal, noise- free simulator so the output distribution can be used as a reference to provide the true output distribution.
- Noisy Simulation: Use a noise model (similar to IBM/Google device parameters) to apply errors, and record the noisy output distribution.
- Data Capture: Collect counts from each mode of simulation $P_{\text{true}}(x)$ from the quiet runs and $P_{\text{noisy}}(x)$ from the noisy runs.

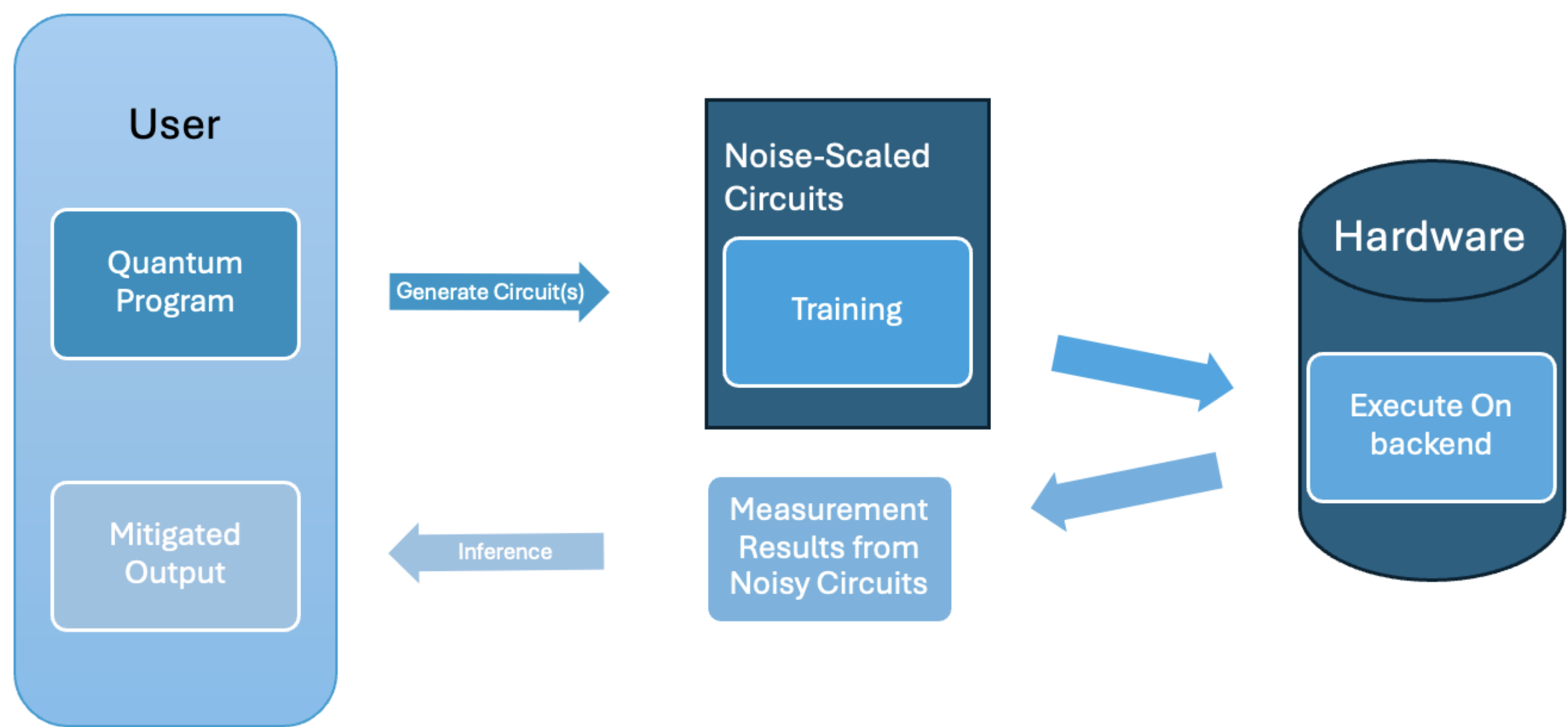
HAMMER: Hamming Reconstruction:

- Step 1: Repeatedly run the circuit; recording counts of all outputs $P(x)$. Step 2: For every candidate x build a Hamming spectrum (counts at distance 0,1,2, etc.) Step 3: Calculate Neighborhood Score $S(x)$ based on clustering neighborhood at nearby distances.
- Step 4: Calculate likelihood of getting this result $L(x)=P(x) \times S(x)$.
- Step 5: Choose x with highest $L(x)$ considered as the corrected output.

Q-BEEP: Bayesian Poisson Error Model:

Motivation: HAMMER misses patterns at larger Hamming distances. Model: Suppose the error counts at every distance follow $\text{Poisson}(\lambda)$. Bayesian Update: Initialize λ from device calibration, then update based on observable counts. Graphical model: Each bitstring is a node in an undirected graph connected by a single bit flip; then reweight nodes iteratively. Inference: Choose bitstring with highest posterior probability.

RESULTS



CONCLUSION AND NEXT STEPS

Conclusion: A structured post-processing method can greatly reduce NISQ errors without changing hardware.

Next Steps:

- Expand to include phase- flip and combined error channels.
- Make it real-time and include on-the-fly calibration in order to adapt the priors.
- Move towards implementing this post-processing on-controller, to enable in-flight error correction.

REFERENCES:

Tannu, S., Stein, S., Wiebe, N., & Su, Y. (2022). HAMMER: Boosting fidelity via Hamming reconstruction. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)* (pp. 123–136). ACM.

Stein, S., Wiebe, N., & Lee, G. (2023). Q-BEEP: Bayesian Poisson error mitigation for quantum computing.

Image from IBM Rensselaer Polytechnic Institute